[Nam Le] - Socket Programming - TCP/UDP Sockets & I/O Multiplexing

1. Network fundamentals + Socket API + TCP

1.1 Protocol Independence + OSI

Protocol:
is the agreement on how programs will communicate across a computer network.
Almost network application follow this **Client/Server** model:

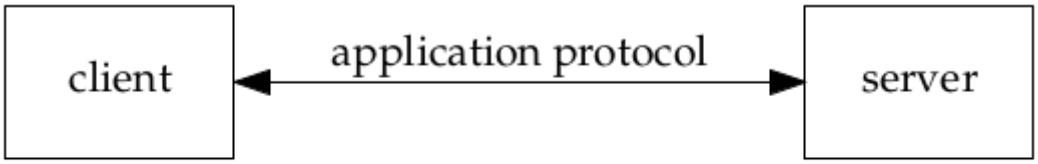


Figure 1.1 Network application: client and server.

OSI & TCP/IP (Internet protocol suite) model:

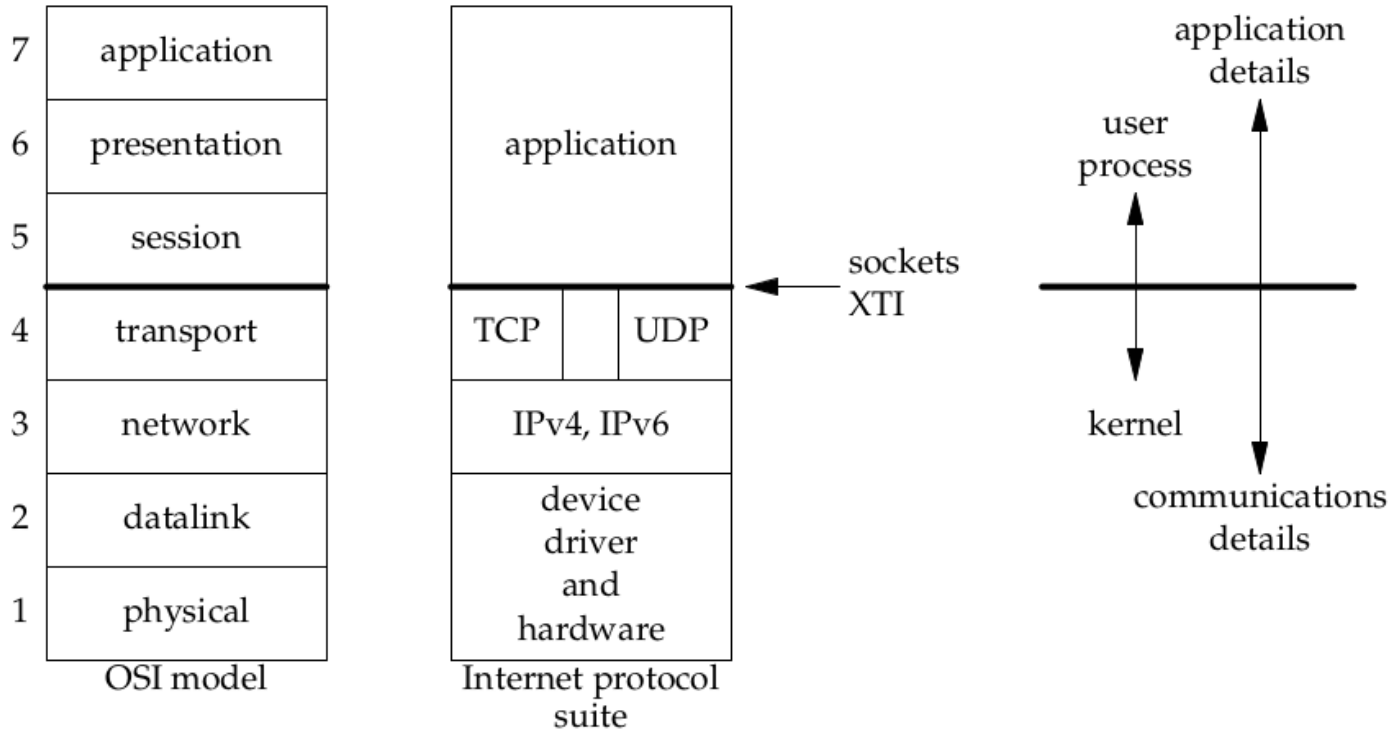


Figure 1.14 Layers in OSI model and Internet protocol suite.

Socket programming are interfaces from the *application* (TCP/IP) into the *transport* layer
Why has this design?

- the upper three layers handle all the details of the application and know little about the communication details. While lower four layers know little about the application, but handle all the communication details
- UNIX provides the separation between *user process* vs *OS kernel*

--> The interface between layer 4 & 5 is the natural place to build the API

BSD (Berkeley Software Distribution) is Operating System, which is the ancestor of Socket API with first is 4.2BSD system

1.2 Socket address structure

```
struct in_addr
{
    in_addr_t s_addr;           /* 32 bits IPv4 address */
};
struct sockaddr_in
{
    sa_family_t sin_family;     /* AF_INET */
```

```
    in_port_t sin_port;         /* 16-bit TCP or UDP port number */
    struct in_addr sin_addr;    /* 32-bit IPv4 address */
    char sin_zero[8];           /* unused */
};
```

Why has `sin_zero`:

- socket API functions like bind() or connect() require a generic address structure (`struct sockaddr`) instead of specific ones. Therefore, we must explicitly `cast` `sockaddr_in` (IPv4) to `sockaddr`
- However, `struct sockaddr` has a fixed size of 16 bytes. Meanwhile, the actual IPv4 data inside `sockaddr_in` (family, port, and IP address) only takes up 8 bytes.
- It acts as an **8-byte padding (filled with zeros)** to bring the total size of sockaddr_in up to exactly **16 bytes**. This ensures that when the kernel reads the structure via the **typecast pointer**, both structures align perfectly in memory
- **BONUS**: When kernel read **2 first bytes** from this structure pointer, it will decide the next (IPv4 or IPv6) action.
  - 2 first bytes is **2 bytes family**

generic socket address:

```
struct sockaddr
  {
    __SOCKADDR_COMMON (sa_);□/* Common data: address family and length.  */
    char sa_data[14];□□/* Address data.  */
  };
```

IPv6 Socket Address Structure:

```
struct in6_addr {
    uint8_t s6_addr[16];   /* 128-bit IPv6 address */
};

struct sockaddr_in6 {
    sa_family_t     sin6_family;    /* AF_INET6 */
    in_port_t       sin6_port;      /* transport layer port# */

    uint32_t        sin6_flowinfo;  /* flow information, undefined */
    struct in6_addr sin6_addr;      /* IPv6 address */

    uint32_t        sin6_scope_id;  /* set of interfaces for a scope */
};
```

1.3 Byte Ordering

Most of developers use the device with litte-edian architechture. However, the embedded device use big-edian architechture. Therefore, if we want send data to another architechture device, we need exchange

Prototype:

```
uint16_t htons(uint16_t host16bitvalue); /* host to network */
uint32_t htonl(uint32_t host32bitvalue);

uint16_t ntohs(uint16_t net16bitvalue);  /* network to host */
uint32_t ntohl(uint32_t net32bitvalue);
```

Example:

```
#include <stdio.h>
#include <stdint.h>
#include <arpa/inet.h>
int main()
{
    uint16_t host_port = 8080;
    uint16_t net_port = htons(host_port);

    printf("Net port: %d\n", net_port);
}
```

Result: `Net port: 36895`
Mathematical formula: **`Result = (Value >> 8) | (Value << 8)`**

EX: 8080 = 0x1F90
0x1F90 >> 8 = 0x001F
0x1F90 << 8 = 0x9000
0x001F | 0x9000 = 0x901F (36895)

1.4 IP Convertion

Prototype:

```
inet_aton(CF, inp);        /* Ascii to Network  */
inet_pton(AF, cp, buf);    /* Convert presentation string of cp (decimal) to network (binary)
inet_ntop(af, cp, buf, len);  /* Network to Presentation */
```

2. TCP socket programing

TCP: Transmission Control Protocol (***byte-stream* protocol**), is connection-oriented protocol providing a reliable, full-duplex byte stream to it users. TCP take care of details such as acknowledgement, timeout, retransmission, ...

- *reliability*: when send data to other end, it requires an ack in return
- *round-trip-time*: algorithms to estimate time between client and server so that it knows how long to wait for an ack
- *sequence*:

TCP contains algorithm to estimate the *round-trip-time* (RTT) between a client and server dynamically so that it knows how long to wait for an acknowledgement.

UDP: User Datagram Protocol, is a connectionless protocol. There is no guarantee that UDP datagrams ever reach their intended destination

- Both can use **either IPv4 or IPv6**

Example about client-server model use TCP :\ `server.c`

```
#include <arpa/inet.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int main () {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    char buffer[1024] = {0};   /*NULL*/

    /* Create Socket*/
    server_fd = socket(AF_INET, SOCK_STREAM, 0); /*Sequenced, reliable, connection-based byte-
stream (TCP)*/
    if (server_fd == -1) printf("Error create socket");
```

```
    address.sin_family = AF_INET;         /* IPv4: 4 bytes for LAN*/
    address.sin_addr.s_addr = INADDR_ANY; /* listen any IP */
    address.sin_port = htons(8080); /* http */

    /* Bind socket into 8080 port */
    if (bind(server_fd, (struct sockaddr *) &address, addrlen) == -1) {
        perror("bind failed");
        exit(1);
    }
    /* Wait for the connection from client */
    if (listen(server_fd, 3) == -1) {
        perror("listen failed");
        exit(1);
    }
    printf("Listen the connection on port 8080...\n");
    while (1) {
        /* accept the connect*/
        new_socket = accept(server_fd, (struct sockaddr*) &address, (socklen_t*)&addrlen);

        /*read data sent form client*/
        read(new_socket, buffer, 1024);
        printf("[Server] Received the message: %s\n", buffer);

        /*Close socket*/
        close(new_socket);
    }
    close(server_fd);

    return 0;
}
```

`client.c`

```
#include <arpa/inet.h>
#include <string.h>
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
int main() {
    int sock = 0;
```

```
    struct sockaddr_in serv_addr;
    char *hello = "Hello from Client!";

    sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock == -1) {
        printf("Error create socket\n");
    }
    else printf("Connect successfully\n");

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(8080);

    /* Convert IP (from decimal to binary)*/
    if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) == -1) {
        printf("Unvalid address\n");
        exit(1);
    }

    /* Connect to server */
    if (connect(sock, (struct sockaddr*) &serv_addr, sizeof(serv_addr)) == -1) {
        printf("Error connect to server\n");
    }

    send(sock, hello, strlen(hello), 0);
    printf("Sent message to server\n");
    close(sock);
    return 0;
}
```

2.1 Threeway Handshake

TCP connection is established:

1. The server must be prepared to accept an incoming connection, done by *socket, bind* and *listen* and is called a *passive open*
2. The client issues an *active* open by calling `connect`. TCP send *SYN* segment (tells the server the client's initial sequence number for the data that the client will send on the connection). **SYN** just contains an **IP header**, **TCP header**, possible **TCP options**

3. Server acknowledge the client's SYN and the server must also its own SYN containing **initial sequence number** for the data that server will send on the connection
4. Client must ack the server's SYN

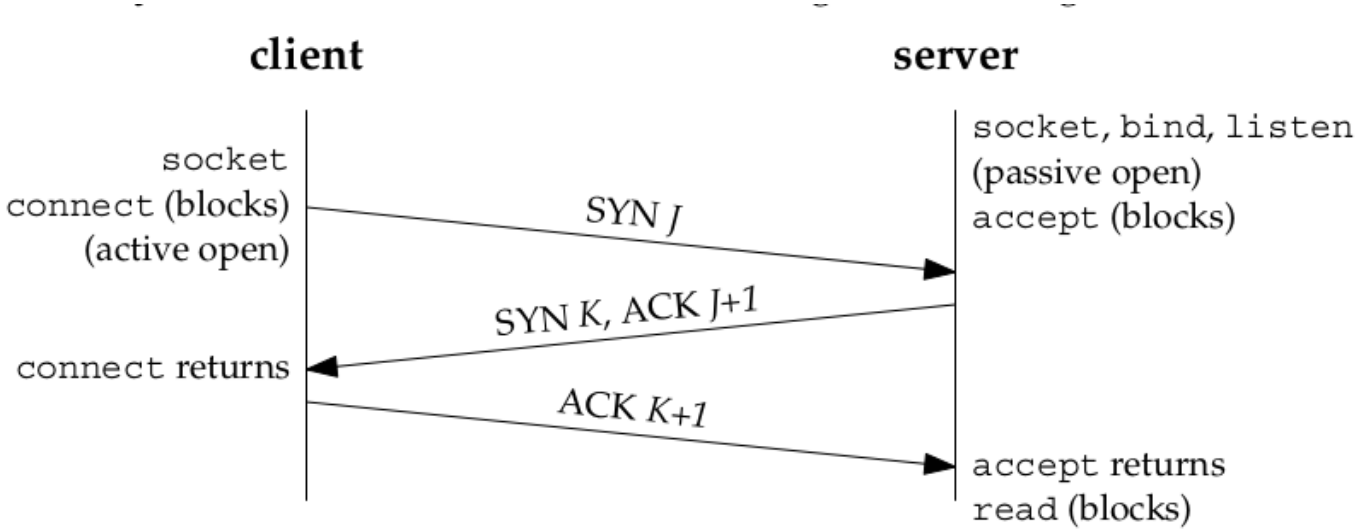


Figure 2.2 TCP three-way handshake.

> **The acknowledgment number in an ACK is the next expected sequence number for the end sending the ACK**

TCP option:

- MSS option (maximum segment size): the max amount data that it's willing to accept in each TCP segment
- Window scale option: (old version 65535 vs new version *almost one gigabyte*)
- Timestamp option: need for high-speed connections to prevent possible data corruption caused by *old, delayed, or duplicated segment*

2.2 TCP termination

1. One app calls *close()* fisrt, we say that this end perform the *active close*. This end's TCP send a **FIN** segment, which means it's finished sending data
2. The other end that receives the FIN performs the *passive close*. The received FIN is **acknowledged by TCP**. **The receipt of FIN is also passed to the application as an end-of-file** (for data may be queued for application to receive).
3. **Sometime later**, application that received **end-of-file** will *close* its socket. That caused its TCP **send a FIN**
4. TCP on system receives this final FIN ack the FIN

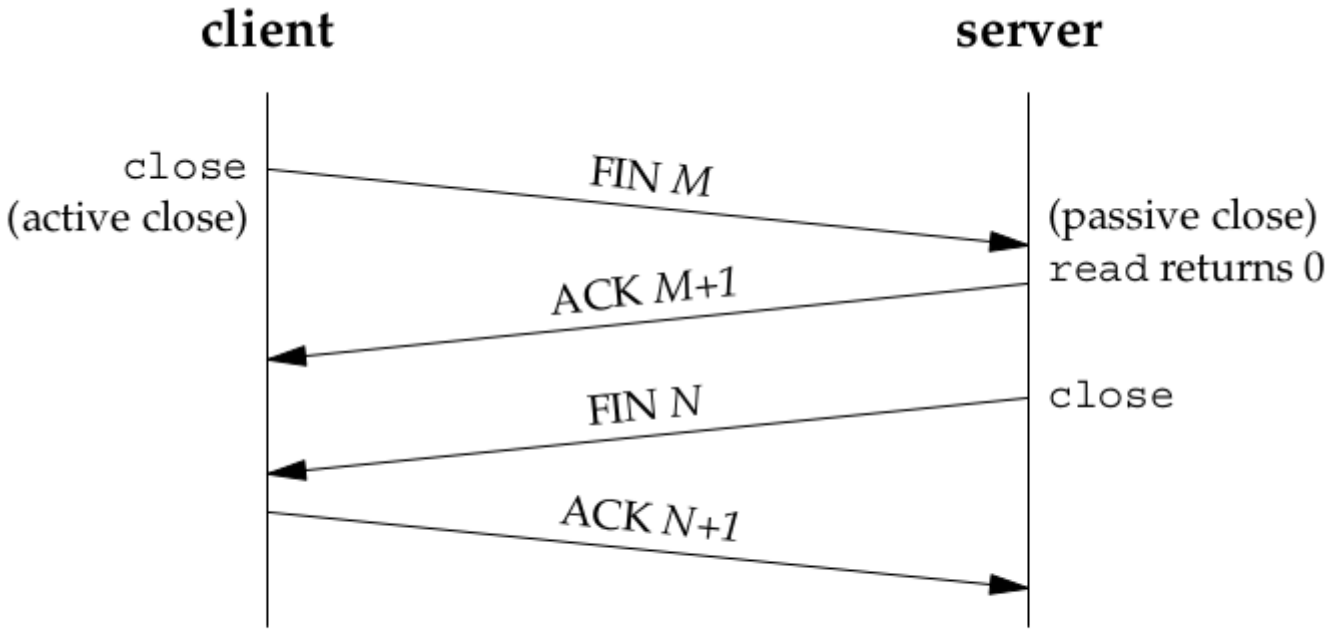


Figure 2.3 Packets exchanged when a TCP connection is closed.

2.3 Echo Server/Client

Steps for complete TCP Server/Client:

1. The client reads a line of text from its standard input and writes the line to the server.
2. The server reads the line from its network input and echoes the line back to the client.
3. The client reads the echoed line and prints it on its standard output.

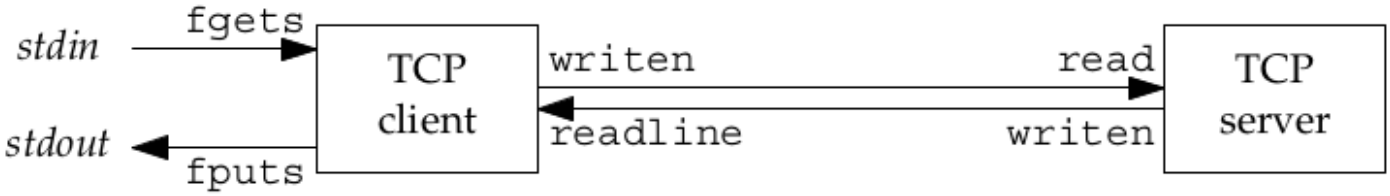


Figure 5.1 Simple echo client and server.

Example: `server.c`

```
#define PORT 8080
#define BUF_SIZE 1024

int main() {
    int server_fd, client_fd;
```

```
    struct sockaddr_in address;
    socklen_t addrlen = sizeof(address);
    char buffer[BUF_SIZE];

    server_fd = socket(AF_INET, SOCK_STREAM, 0);    // Create socket
    if (server_fd == -1) { perror("socket"); exit(1); }

    int opt = 1;                /* config option for socket */
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR,             /* SOL_SOCKET: basic lever of
socket*/
            &opt, sizeof(opt));         /* SO_RESEADDR: bind close port at the moment, no
TIME_WAIT  */

    memset(&address, 0, sizeof(address));
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr*)&address, sizeof(address)) == -1) {
        perror("bind"); exit(1);     /* bind the address to server fd*/
    }
    if (listen(server_fd, 3) == -1) {
        perror("listen"); exit(1);  /* listen for connection*/
    }
    printf("Echo server listening on port %d...\n", PORT);

    while (1) {
        client_fd = accept(server_fd, (struct sockaddr*)&address, &addrlen);
        if (client_fd == -1) { perror("accept"); continue; }
        printf("Client connected\n");

        ssize_t n;
        while ((n = read(client_fd, buffer, BUF_SIZE - 1)) > 0) {
            buffer[n] = '\0';
            printf("Received: %s", buffer);
            write(client_fd, buffer, n);   // echo again
        }
        printf("Client disconnected\n");
        close(client_fd);
    }
```

```
    close(server_fd);
    return 0;
}
```

client.c

```
#define PORT 8080
#define BUF_SIZE 1024

int main() {
    int sock;
    struct sockaddr_in serv_addr;
    char buffer[BUF_SIZE];

    sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock == -1) { perror("socket"); exit(1); }

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    /*Convert presentation (decimal) to network (binary) */
    if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <= 0) {
        printf("Invalid address\n"); exit(1);
    }

    if (connect(sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) == -1) {
        perror("connect"); exit(1);
    }

    while (1) {
        printf("Enter message (or 'exit'): ");
        if (!fgets(buffer, BUF_SIZE, stdin)) break;
        if (strncmp(buffer, "exit", 4) == 0) break;

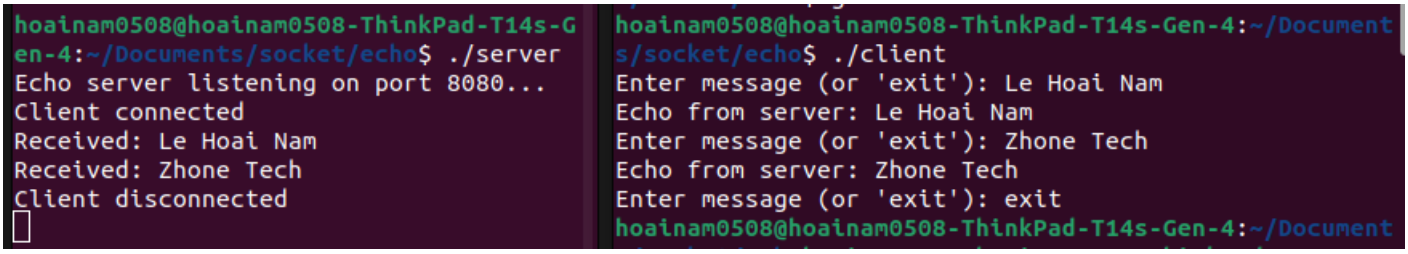
        write(sock, buffer, strlen(buffer));

        int n = read(sock, buffer, BUF_SIZE - 1);
        if (n <= 0) { printf("Server closed connection\n"); break; }
        buffer[n] = '\0';
```

```
        printf("Echo from server: %s", buffer);
    }

    close(sock);
    return 0;
}
```

Result:



2.4 getsockname & getpeername

`getsockname()` --> get local IP and local Port
`getpeername()` --> get peer/remote IP and Port

Example: getsockname()

```
/* After Client successfully connect to socket */

/* server.c */
int server_fd, client_fd;
struct sockaddr_in address;
socklen_t addrlen = sizeof(address);
/* Create socket */
/* Bind address */
/* Listen connection */
client_fd = accept(server_fd, (struct sockaddr*)&address, &addrlen);

if (getsockname(client_fd, (struct sockaddr*)&address, &addrlen) == 0){
    printf("My local IP: %s\n", inet_ntoa(address.sin_addr));
    printf("Local Port: %d\n", ntohs(address.sin_port));
}
```

Result:

```
Client connected
My local IP: 127.0.0.1
Local Port: 8080
```

> Similar for `getpeername()` but connect to remote/peer connection

2.5 Concurrent TCP Server (Reality)

method: use *fork* a child process to handle each client
Example:

```
pid_t pid;
int listenfd, connfd;
listenfd = Socket( ... );
/* fill in sockaddr_in{} with server's well-known port */
Bind(listenfd, ... );
Listen(listenfd, LISTENQ);
for ( ; ; ) {
connfd = Accept(listenfd, ... );                    /* probably blocks */
if ( (pid = Fork()) == 0) {
Close(listenfd);        /* child closes socket */
doit(connfd);           /* process the request */
Close(connfd);          /* child close connection */
exit(0);                /* child terminates */
}
Close(connfd);          /* parent closes connection */
}
```

Idea:



Figure 4.14 Status of client/server before call to `accept` returns.

Then, `accept` return

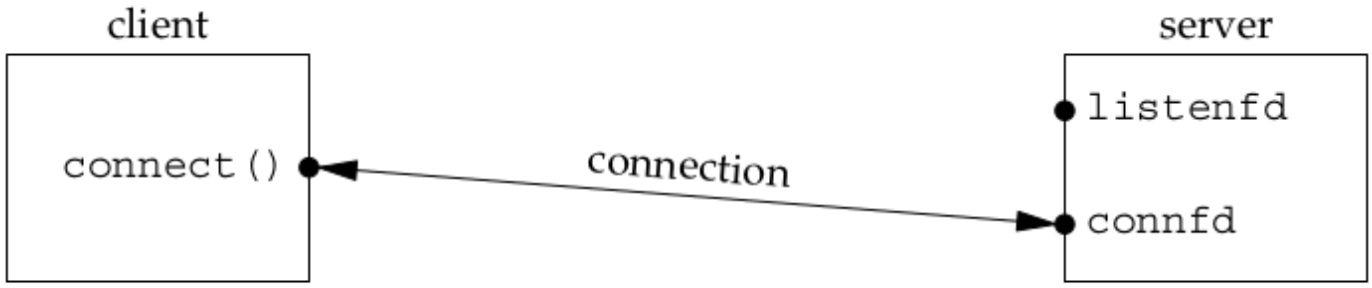


Figure 4.15 Status of client/server after return from `accept`.

After fork, child and parent both hold the shared (duplicated) descriptor

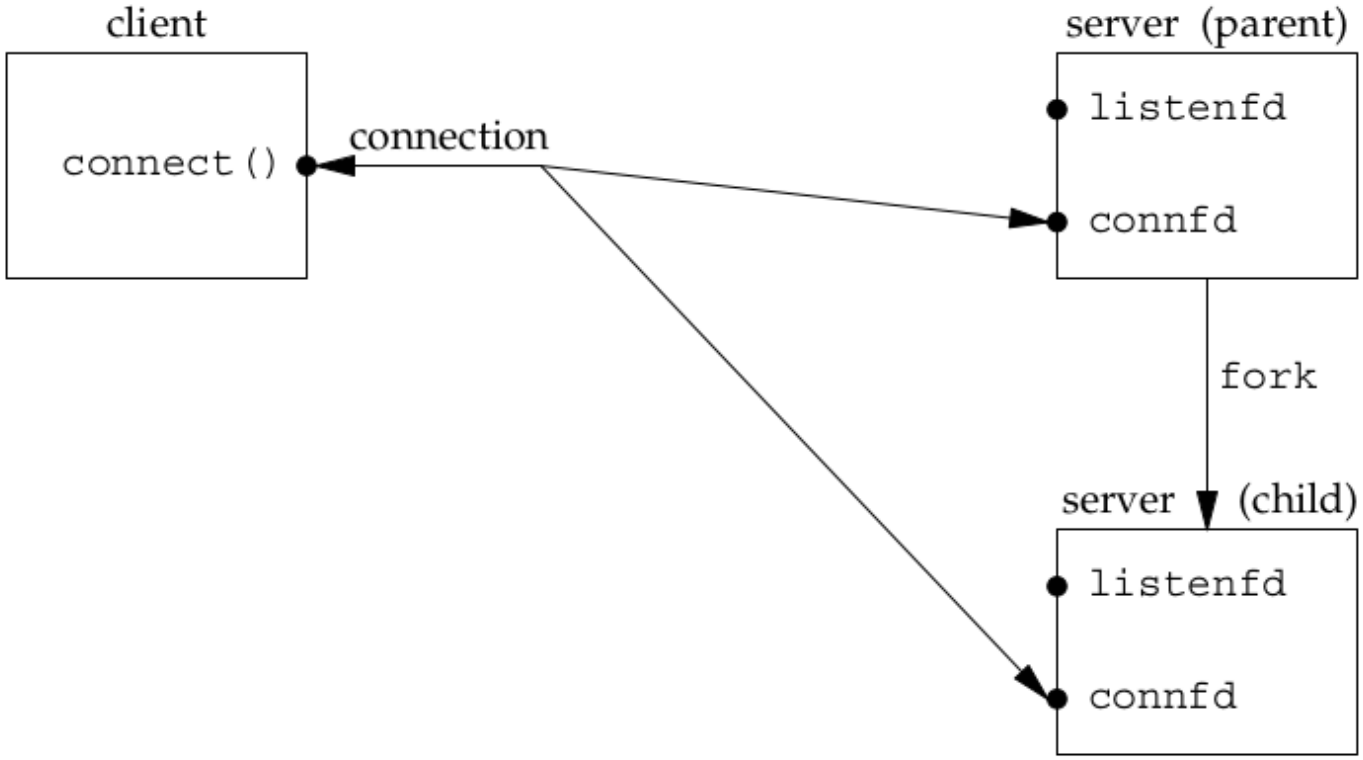
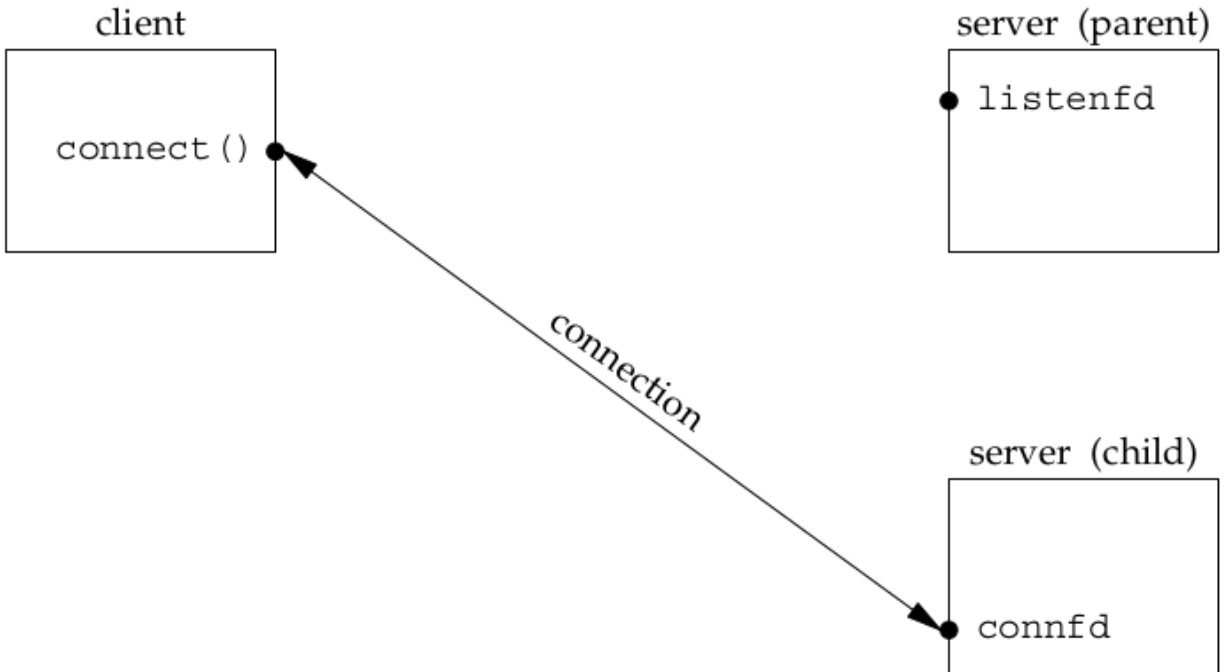


Figure 4.16 Status of client/server after `fork` returns.

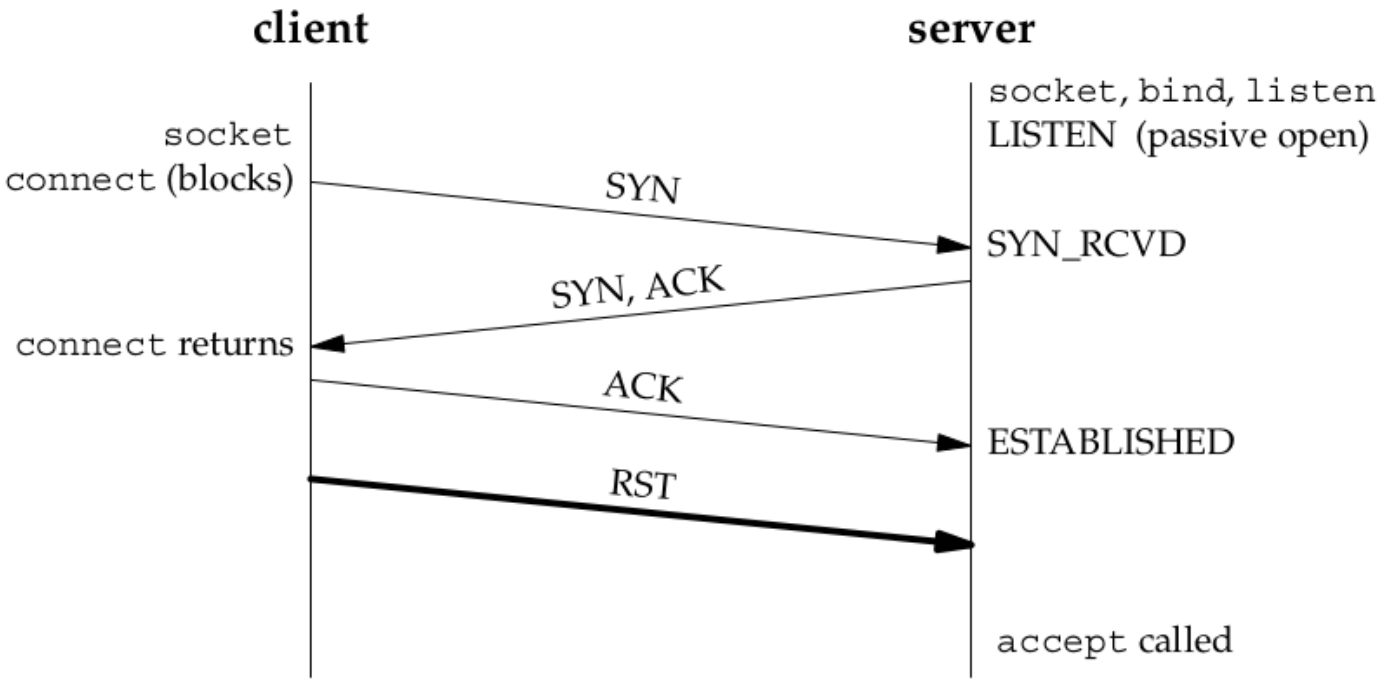
parent close *connfd* (from client), child close *listenfd* (socket)



2.6 TCP termination

We can use Control D to **Enf-of-File** (EOF character) --> then terminates the client --> the client enters the TIME_WAIT state

Connection Abort before accept Returns



We can see this picture
Normally, we only establish three-way handshake between server and client and then we *accept* is called immediately. But this pic show that, to *accept* is called, TCP send **RST**.
The reason is, sometimes connection may be **abort**, so we cant transmit data between server and client. This time, TCP only call **RST** (reset) to reconnect the connection between these.
Unfortunately, abort connection is implementation-dependent, its mean that this error will be handle within **kernel**, server process never sees it.
--> we only know the error will return form `accept()` and the error depends on the implementation

SIGPIPE vs SIGCHLD signal

Feature	SIGCHLD	SIGPIPE
Full name	Signal Child	Signal Pipe
Occur when?	Child, is created by fork() terminate, stop or resume.	Process try to write, send to a Pipe/Socket which close in another fd.
Default Result	Ignore. Parent continue run normally.	Terminate immediately if no handle or ignore.

Feature	SIGCHLD	SIGPIPE
Consequence if no handle	Child become Zombie Process if parent doesn't call wait() / waitpid().	Program is crash immediately if try to write close pipe/socket.

Simulation SIGPIPE:



3. UDP (User Datagram protocol)

- Client does not establish a connection with server. Just send datagram to the server
  - using `sendto` function
- Server does not accept a connection from client
  - just call `recvfrom` which wait until data arrives from some client

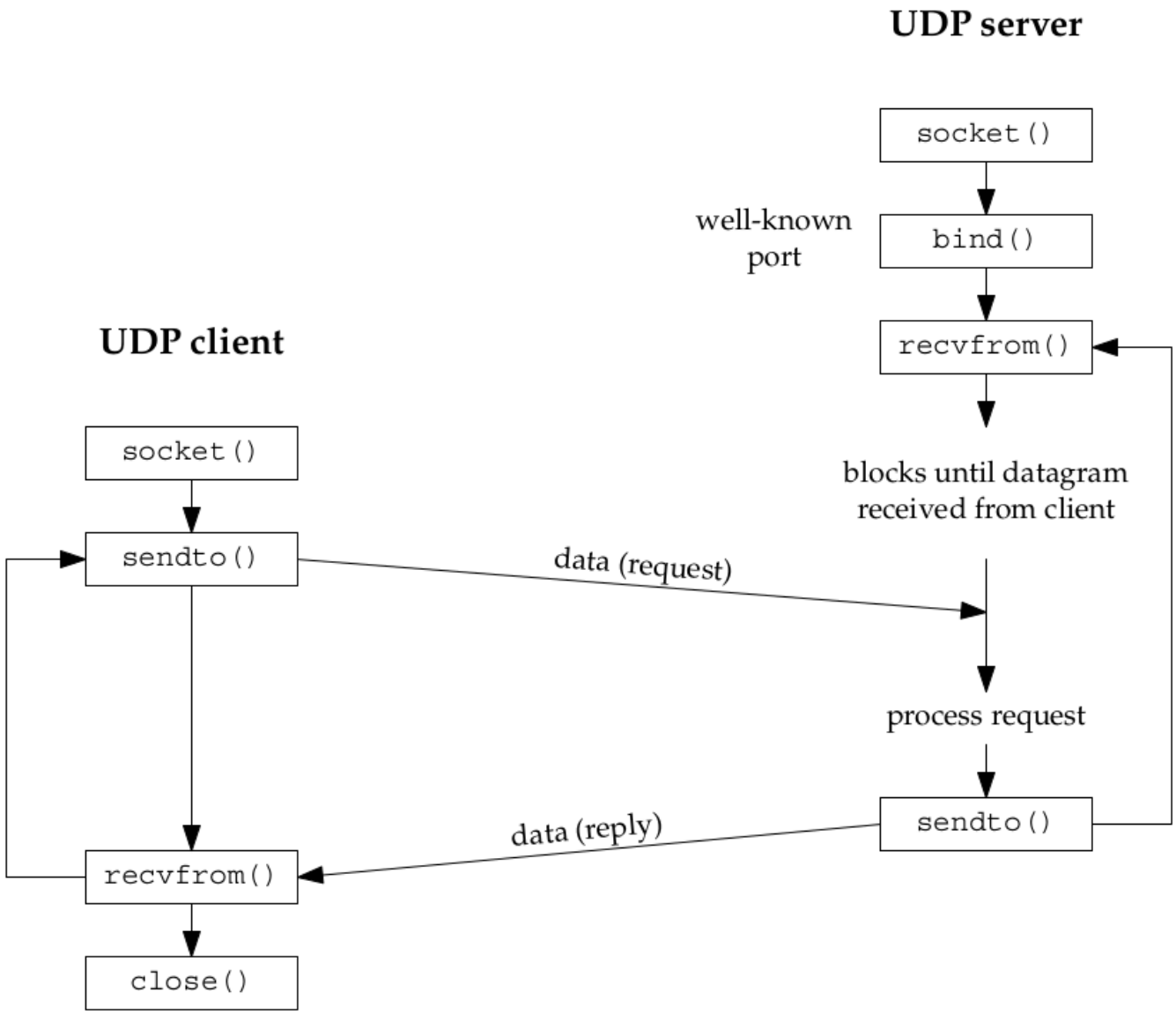


Figure 8.1 Socket functions for UDP client/server.

\

Example UDP client/server (echo, verrying received respone)

`connect` **function with UDP** As we know, use `send_to` function in client slide may be occur asynchoronous (if server is not running, while `sendto` function in client still running --> sendto is blocked)
To deal this solution, we can use `connect` function. But:

- no have result like TCP connecttion, no three-way handshake
- Instead, kernel will check immediately errors (ex Unreachable Destination)
  - records the IP addr and PORT number of the peer (from the socket address structure passed to `connect`)

We must do these if use `connect`:

- use `write` or `send` instead of `sendto`

- use `read` `recv` or `recvmsg` instead `recvfrom`
- connected UDP (or `connect`) exchange datagrams with only one IP addr

> UDP client or server can call connect only if that process uses the UDP socket to communicate with exactly one peer.

Calling *connect* Multiple Times for a UDP Socket
A process with a connected UDP socket can call connect again for that socket for one of two reasons:

- To specify a new IP address and port
- To unconnect the socket

Lack of flow control:

```
for (int i = 0; i < 2000; i++) {
    sendto(sockfd, sendline, strlen(sendline), 0, (struct sockaddr*)&serv_addr, addrlen);
}
```

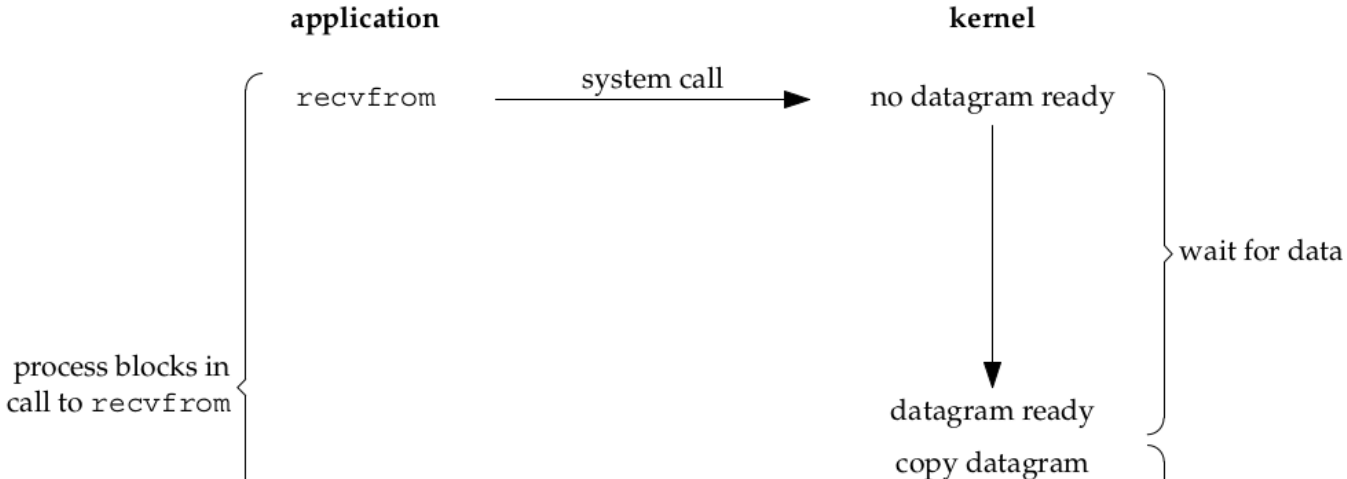
Result from server: `Number of datagrams is received is 1666`
The reason: UDP not having any flow control

4. I/O Multiplexing

What is I/o Multiplexing?
Assump the sistuation that client is blocked in a call to `fgets` and the server process was kill. Then server TCP correctly sent a FIN to the client TCP, but since the client process was blocked reading from stdin, it never saw the **EOF** until it read from the socket (much later).
--> We need the capability to tell kernel that we want to be notified if one or more I/O conditions are ready --> **This is I/O Multiplexing_**

I/O Models:

Blockking model



`recvfrom` does not return until data arrives and copied into our application buffer, or an error occur. We say that out process is *blocked* the entire time from when it calls *recvfrom* until it returns

NonBlocking Model meaning: Instead of putting the process to keep, Let return an error!

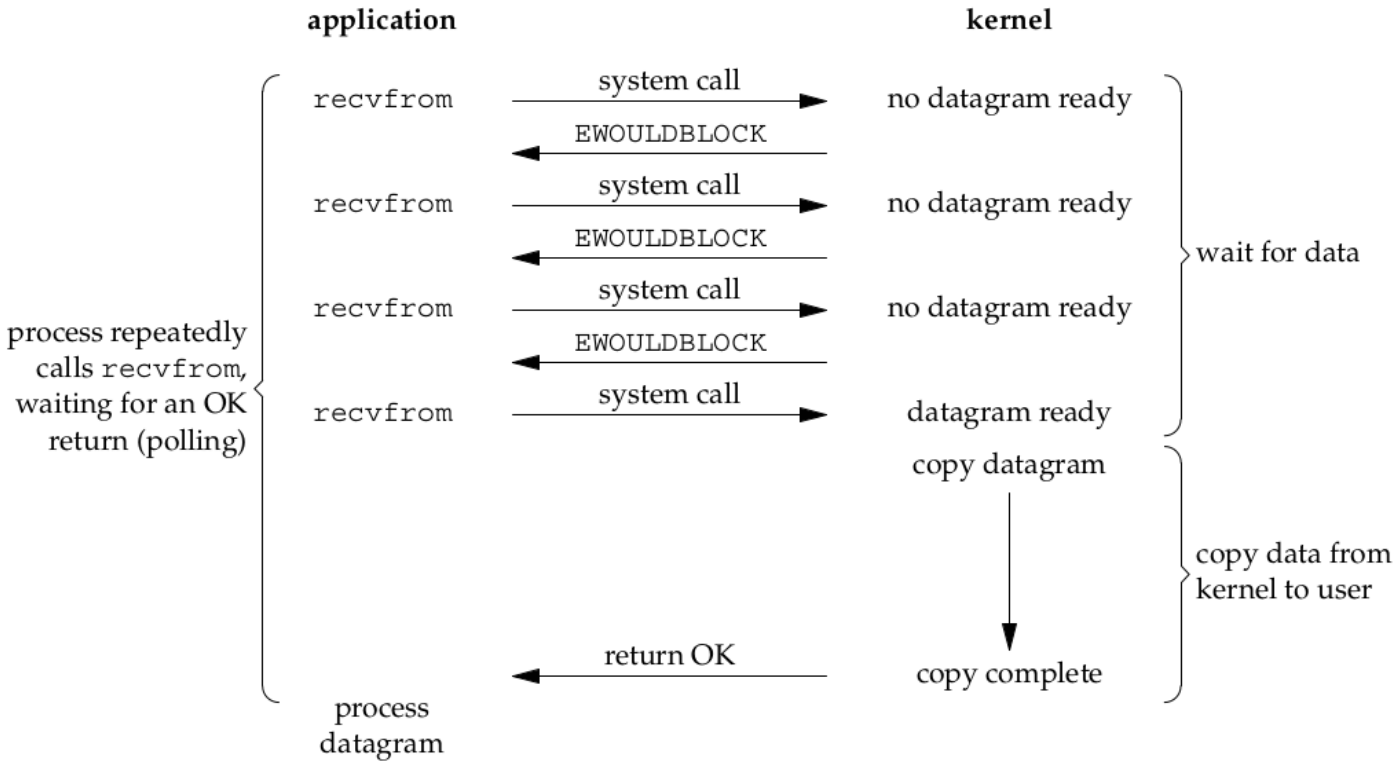


Figure 6.2 Nonblocking I/O model.

I/O Multiplexing Model

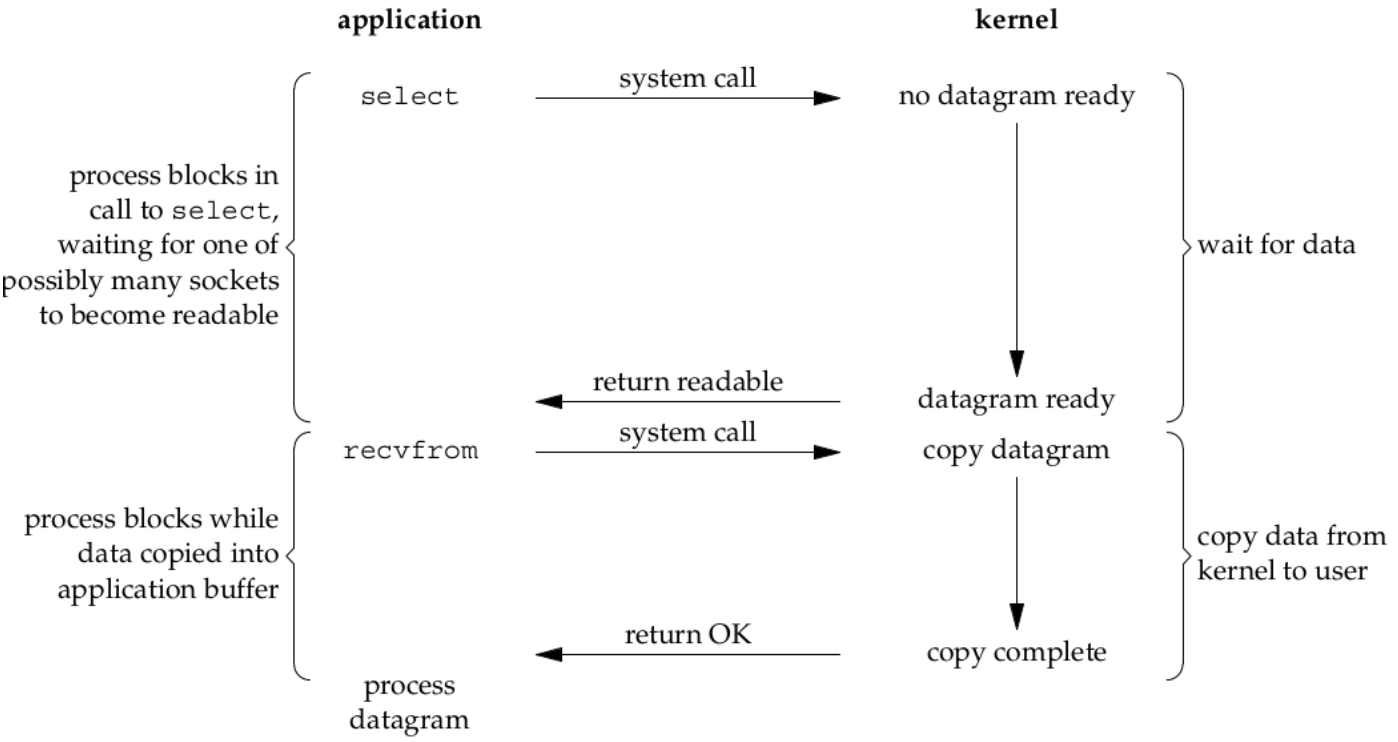


Figure 6.3 I/O multiplexing model.

`select` function

allow kernel instruct the kernel to wait for any one of multiple events to occur , and
to wake up the process only when one or more of these events occurs , or when a specified amount of time has passed
Protocol:

```
#include <sys/select.h>
#include <sys/time.h>
int select(int maxfdp1, fd_set *readset, fd_set *writeset, fd_set *exceptset,const struct
timeval *timeout);
              /* Returns: positive count of ready descriptors, 0 on timeout, –1 on error */

struct timeval {
long tv_sec;    /* second */
long tv_usec;    /* microsecond */
};
```

`pselect` function

invented by POSIX, contains 2 changes from `select` function

```
#include <sys/select.h>
#include <signal.h>
#include <time.h>
int pselect(int maxfdp1, fd_set *readset, fd_set *writeset, fd_set *exceptset,
const struct timespec *timeout, const sigset_t *sigmask);
                            /* Returns: count of ready descriptors, 0 on timeout, –1 on error
*/
```

- use *timespec* structure, not *timeval*

  ```
  struct timespec {
  time_t tv_sec;    /* second */
  long  tv_nsec;    /* nanosecond */
  };
  ```

- have `sigmask` argument: allows the program to disable the delivery of certain signals, test some global variables that are set by the handlers for these new-disabled signals

`poll` function:

```
#include <poll.h>
int poll(struct pollfd *fdarray, unsigned long nfds, int timeout);
```

```
/* Returns: count of ready descriptors, 0 on timeout, –1 on error */



struct pollfd {
int fd;            /* descriptor to check */
short events;      /* events of interest on fd */
short revents;     /* events that occurred on fd */
};
```

Lab 4. Chat program use `select()`

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/select.h>

#define PORT      8080
#define MAXLINE   1024
#define MAXCLI    FD_SETSIZE          /* giới hạn 5 client */

int main() {
    int sockfd, client_fd, conn_fd;
    int client[MAXCLI];      /* fd c□a từng client, -1 = tr□ng */
    int nclients = 0;        /* s□ client hiện tại */
    int maxi = -1;
    int maxfd;
    fd_set rset, allset;
    int i;

    struct sockaddr_in serv_addr, cli_addr;
    socklen_t clilen;
    char buf[MAXLINE];
    ssize_t n;

    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd == -1) { perror("socket"); exit(1); }

    int opt = 1;
```

```
    setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    memset(&serv_addr, 0, sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_addr.s_addr = INADDR_ANY;   /* l□ng nghe trên mọi IP c□a máy, k□ c□ IP LAN
*/
    serv_addr.sin_port = htons(PORT);

    if (bind(sockfd, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) == -1) {
        perror("bind"); exit(1);
    }
    if (listen(sockfd, MAXCLI) == -1) {
        perror("listen"); exit(1);
    }

    for (i = 0; i < MAXCLI; i++) client[i] = -1;

    maxfd = sockfd;
    FD_ZERO(&allset);
    FD_SET(sockfd, &allset);

    printf("Chat server listening on port %d (max %d clients)...\n", PORT, MAXCLI);

    while (1) {
        rset = allset;
        int nready = select(maxfd + 1, &rset, NULL, NULL, NULL);
        if (nready < 0) { perror("select"); continue; }

        if (FD_ISSET(sockfd, &rset)) {
            clilen = sizeof(cli_addr);
            client_fd = accept(sockfd, (struct sockaddr*)&cli_addr, &clilen);
            printf("New connection from %s:%d, fd=%d\n",
                    inet_ntoa(cli_addr.sin_addr), ntohs(cli_addr.sin_port), client_fd);

            for (i = 0; i < MAXCLI; i++) {
                if (client[i] < 0) { client[i] = client_fd; break; }
            }
            if (i > maxi) maxi = i;
            nclients++;
```

```
            FD_SET(client_fd, &allset);
            if (client_fd > maxfd) maxfd = client_fd;

            /* thông báo cho các client khác có người mới vào */
            char joinmsg[MAXLINE];
            snprintf(joinmsg, sizeof(joinmsg), "*** Client fd=%d joined the chat ***\n",
client_fd);
            for (int j = 0; j <= maxi; j++) {
                if (client[j] >= 0 && client[j] != client_fd) {
                    write(client[j], joinmsg, strlen(joinmsg));
                }
            }
            if (--nready <= 0) continue;
        }

        /* 2. Duyệt các client đang m */
        for (i = 0; i <= maxi; i++) {
            if ((conn_fd = client[i]) < 0) continue;

            if (FD_ISSET(conn_fd, &rset)) {
                n = read(conn_fd, buf, MAXLINE - 1);
                if (n <= 0) {
                    printf("Client fd=%d disconnected\n", conn_fd);
                    close(conn_fd);
                    FD_CLR(conn_fd, &allset);
                    client[i] = -1;
                    nclients--;

                    char leftmsg[MAXLINE];
                    snprintf(leftmsg, sizeof(leftmsg), "*** Client fd=%d left the chat ***\n",
sockfd);
                    for (int j = 0; j <= maxi; j++) {
                        if (client[j] >= 0) write(client[j], leftmsg, strlen(leftmsg));
                    }
                } else {
                    buf[n] = '\0';
                    printf("[fd=%d] %s", conn_fd, buf);

                    /* BROADCAST: g□i cho mọi client khác, trừ người g□i */
                    char outbuf[MAXLINE + 32];
```

```
                    snprintf(outbuf, sizeof(outbuf), "[fd=%d]: %s", sockfd, buf);

                    for (int j = 0; j <= maxi; j++) {
                        if (client[j] >= 0 && client[j] != conn_fd) {
                            write(client[j], outbuf, strlen(outbuf));
                        }
                    }
                }

                if (--nready <= 0) break;
            }
        }
        /* sockfd bị dùng làm biến lặp trong for trên, listenfd không đổi nên không ảnh hưởng vòng while sau */
    }

    close(sockfd);
    return 0;
}
```

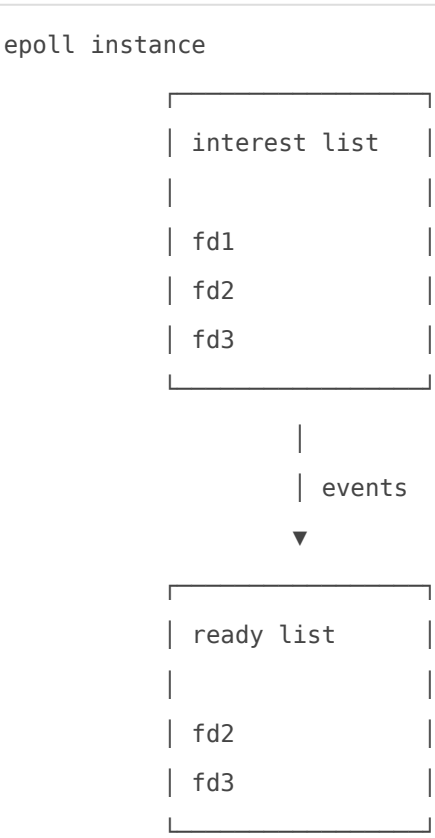
5. epoll API

Like the I/O multiplexing system calls, the Linux `epoll` (event poll) API is used to monitor multiple file descriptors to see if they are ready for I/O.

The `epoll` API appear in Linux 2.6.
The central data structure of the epoll API is an **epoll instance**, which is reffered to via an open file descriptor.
This file descriptor is not used for I/O. Instead, it is a handle for kernel data structures that serve 2 purposes:

- **interest list**: recording a list of file desriptors that this process has declared an interest in monitoring
- **ready list**: maintaing a list of file that are ready for I/O

> The membership of the ready list is a subset of the interest list



The `epoll_create()` system call creates a new epoll instance whose interest list is initially empty.

```
#include <sys/epoll.h>
int epoll_create(int size);
                                /* Returns file descriptor on success, or –1 on error */
```

the *size* argument is not an upper limit, but rather a **hint** to kernel about how to initially dimension internal data structure --> not hard limit and we can change this limit by *epoll_ctl*
But from Linux 2.6.8: **epoll use *red-black tree* to monitor the file descriptors** (self-balance and self-resize) --> then `size` is ignored

Then they provide new function:

```
#include <sys/epoll.h>
int epoll_create1(int flag);
                                /* Returns file descriptor on success, or –1 on error */
```

Support only one flag: **EPOLL_CLOEXEC**. It mean if we call this flag, file descriptor will be close before `exec()` run --> to ensure no **race condition** between parent and child process

The `epoll_ctl()` system call modifies the interest list of the epoll instance referred to by the file descriptor *epfd*

```
#include <sys/epoll.h>
int epoll_ctl(int epfd, int op, int fd, struct epoll_event *ev);
                                                    /* Returns 0 on success, or –1 on error */
```

We can:

- add the file decriptor into interest list (with set of event)
- change the event of file descriptor already exist in interest list
- delete file descriptor from interest list

The epoll_wait() system call returns information about ready file descriptors from the epoll instance referred to by the file descriptor epfd

```
#include <sys/epoll.h>
int epoll_wait(int epfd, struct epoll_event *evlist, int maxevents, int timeout);
                              /* Returns number of ready file descriptors, 0 on timeout, or –1
on error */
```

Level Triggered vs Edge Triggered

- Level-Triggered: A file descriptor is considered to be ready if it is possible to perform an I/O system call without blocking
- Edge-Triggered: Notification is provided if there is I/O activity on afile descriptor since it was last monitored.

Revision #7
Created 2026-09-23 08:14:58 UTC by Lê Hoài Nam
Updated 2026-09-30 07:40:33 UTC by Lê Hoài Nam